

Как это сделано в настоящем Bash

Источник: GNU Bash, коммит 9c46586 от 15.09.2026. Полные исходники:

```
git clone https://git.savannah.gnu.org/git/bash.git
```

Код на C, но знать C не нужно: фрагменты короткие, ключевые строки прокомментированы.

Комментарии в `struct builtin` переведены и сокращены, код не менялся.

Фрагмент А. Как в Bash добавляется команда

```
/* builtins.h, строка 53. Так Bash описывает любую встроенную команду.
   Комментарии переведены и сокращены. */
struct builtin {
    char *name; /* имя, которое набирает пользователь */
    sh_builtin_func_t *function; /* функция, которая ее выполняет */
    int flags;
    char * const *long_doc; /* текст для help */
    const char *short_doc;
    char *handle;
};

/* examples/loadables/hello.c. Команду можно собрать отдельно
   и подгрузить в уже работающий bash: enable -f ./hello.so hello */
int
hello_builtin (WORD_LIST *list) /* у всех команд одна сигнатура:
                                   список слов на вход, код выхода на выход */
{
    printf("hello world\n");
    fflush (stdout);
    return (EXECUTION_SUCCESS);
}

struct builtin hello_struct = {
    "hello", /* builtin name */
    hello_builtin, /* function implementing the builtin */
    BUILTIN_ENABLED, /* initial flags for builtin */
    hello_doc, /* array of long documentation strings. */
    "hello", /* usage synopsis; becomes short_doc */
    0 /* reserved for internal use */
};
```

Встроенные команды (echo , cd , pwd и еще около 70) описаны в файлах builtins/*.def , по одной или несколько в файле. Из них при сборке генерируется общая таблица, и Bash ищет в ней команду по имени.

Вопрос: сколько мест надо поменять, чтобы добавить команду в Bash? А в нашем шелле?

Фрагмент В. Как Bash исполняет конструкции языка

`/* execute_cmd.c, функция execute_command_internal, около 630 строк.`

`Здесь только скелет. command->type это вид конструкции: for, if, while... */`

```
switch (command->type)
{
  case cm_simple:    { /* около 115 строк: простая команда вроде ls -l */ } break;
  case cm_for:       exec_result = execute_for_command (command->value.For); break;
  case cm_arith_for: exec_result = execute_arith_for_command (command->value.ArithFor);
  case cm_case:      exec_result = execute_case_command (command->value.Case); break;
  case cm_while:     exec_result = execute_while_command (command->value.While); break;
  case cm_if:        exec_result = execute_if_command (command->value.If); break;
  /* ... еще 7 веток */
}
```

`/* print_cmd.c: такой же switch, чтобы напечатать конструкцию */`

```
switch (command->type)
{
  case cm_for: print_for_command (command->value.For); break;
  /* ... */
}
```

`/* copy_cmd.c и dispose_cmd.c: тот же switch еще два раза,
чтобы скопировать конструкцию и освободить память */`

Вопрос: в нашем монолите тоже есть цепочка if, только по имени команды. Чем эти два случая отличаются? Подсказка: что в Bash меняется чаще, набор команд или набор конструкций языка вроде `for` и `if`?

★ Фрагмент С

Эти места стоит открыть в полных исходниках.

execute_cmd.c , строка 219:

```
volatile int last_command_exit_value;
```

Код завершения последней команды хранится в глобальной переменной. Она встречается около 150 раз в 20 файлах. Какой это запах, и почему в C-программе 1989 года так сделали?

variables.c , строка 424:

```
parse_and_execute (temp_string, tname, SEVAL_NONINT | SEVAL_NOHIST | SEVAL_FUNCDEF | SEVAL_ON
```

Так Bash импортирует функции из переменных окружения: строка передается в общий `parse_and_execute` , который умеет разобрать и выполнить любую команду. Флаги `SEVAL_FUNCDEF` («только определение функции») и `SEVAL_ONECMD` («только одна команда») появились в патче после уязвимости Shellshock в 2014 году. Что было не так с дизайном до патча?